

# Robot State Estimation

---

## Robot State Estimation

1. Course Content
2. Preparations
  - 2.1 Content Description
  - 2.2 Start the Agent
3. Run the Example
  - 3.1 Start the Program
  - 3.2 View Fused Odometry Data
4. Source Code Explanation
5. Fused Odometry Parameters

## 1. Course Content

---

Basic: After the program runs, it combines the IMU data read from the ROS expansion board for multi-sensor fusion to obtain fused odometry data.

Advanced: Understand the meaning of `robot_localization` parameters and the principle of odometry fusion.

## 2. Preparations

---

### 2.1 Content Description

This lesson uses the Jetson Orin NX as an example. For Raspberry Pi boards, you need to open a terminal on the host machine, enter the command to get into the Docker container, and then input the commands mentioned in this lesson in the terminal within the Docker container. A tutorial on how to enter the Docker container from the host can be found in this product's guide [00.Configuration and Operation Guide]-[3.Enter the Docker (For RPI5)]. For ORIN and RDK boards, you can directly open a terminal and enter the commands mentioned in this lesson.

### 2.2 Start the Agent

**Note:** If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
mircoROS_Agent
subscriber created | client_key: 0x1D229138, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1764917701.432450] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1764917701.436381] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x1D229138, topic_id: 0x006(2), participant_
id: 0x000(1)
[1764917701.439564] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x1D229138, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1764917701.444080] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1764917701.449158] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x1D229138, topic_id: 0x007(2), participant_
id: 0x000(1)
[1764917701.452629] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x1D229138, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1764917701.457477] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

## 3. Run the Example

### 3.1 Start the Program

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Enter the command,

```
ros2 launch m3_bringup car_base.launch.py
```

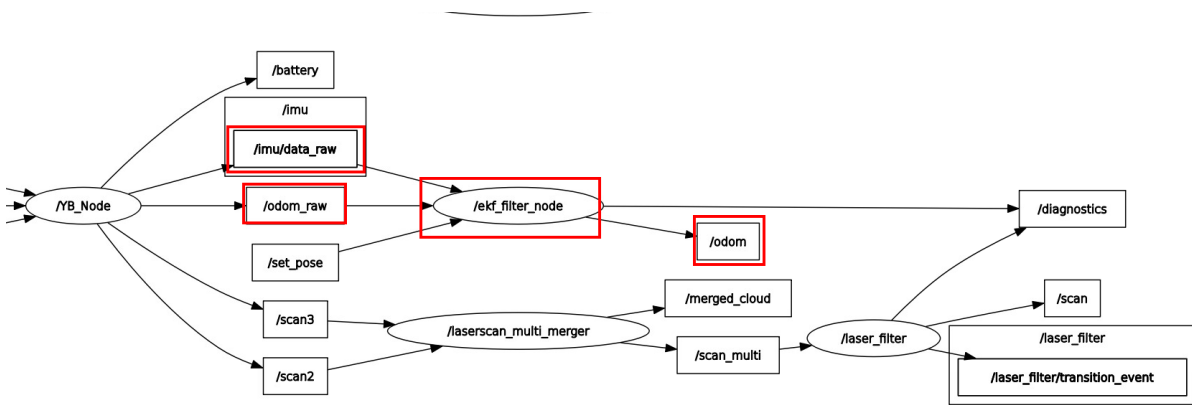
```

jetson@yahboom: ~
jetson@yahboom:~$ start_agent
jetson@yahboom:~$ ros2 launch m3_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-12-05-16-21-12-412067-yahboom-126288
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: =====Robot model ROSMASTER-M3 start=====
[INFO] [launch.user]: =====LASER_TYPE:single=====
[INFO] [launch.user]: LASER start sucefully!,Start by independent node
[INFO] [launch.user]: =====Robot Base Start=====
[INFO] [joint_state_publisher-1]: process started with pid [126344]
[INFO] [robot_state_publisher-2]: process started with pid [126346]
[INFO] [lasers_merger-3]: process started with pid [126348]
[INFO] [scan_to_scan_filter_chain-4]: process started with pid [126350]
[robot_state_publisher-2] [INFO] [1764922872.990533755] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1764922872.990734044] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1764922872.990757373] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-2] [INFO] [1764922872.990768989] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-2] [INFO] [1764922872.990778493] [robot_state_publisher]: got segment laser23_frame
[robot_state_publisher-2] [INFO] [1764922872.990788029] [robot_state_publisher]: got segment lwheel1_Link
[robot_state_publisher-2] [INFO] [1764922872.990797373] [robot_state_publisher]: got segment lwheel2_Link
[robot_state_publisher-2] [INFO] [1764922872.990806333] [robot_state_publisher]: got segment rwheel1_Link
[robot_state_publisher-2] [INFO] [1764922872.990815549] [robot_state_publisher]: got segment rwheel2_Link
[lasers_merger-3] [INFO] [1764922873.025192319] [laserscan_multi_merger]: Waiting for topics ...
[scan_to_scan_filter_chain-4] [WARN] [1764922873.047688288] [rcl.logging_rosout]: Publisher already registered for provided node name. If this is due to multiple nodes with the same name then all logs for that logger name will go out over the existing publisher. As soon as any node with that name is destructed it will unregister the publisher, preventing any further logs for that name from being published on the rosout topic.
[joint_state_publisher-1] [INFO] [1764922873.716521379] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic...
[lasers_merger-3] [INFO] [1764922874.029006404] [laserscan_multi_merger]: Subscribing to topics 2

```

- There is no visual display here, you can check the relationship between nodes and the published topics in rqt\_graph.

```
ros2 run rqt_graph rqt_graph
```



- The `ekf_filter_node` node receives `odom_raw` data and `imu/data_raw` data for fusion, and finally outputs and publishes `odom` odometry data, where:
  - `/odom_raw` is the wheel odometry data calculated by the chassis's STM32, which collects encoder data from each wheel motor and computes it using the forward kinematics of the Mecanum wheel chassis.
  - `/imu/data_raw` is the data published by the chassis's STM32 after collecting IMU data.
  - The `ekf_filter_node` node is one of the nodes in the `robot_localization` package and can fuse an arbitrary number of sensors. The node has no limit on the number of input sources.

[TIP]

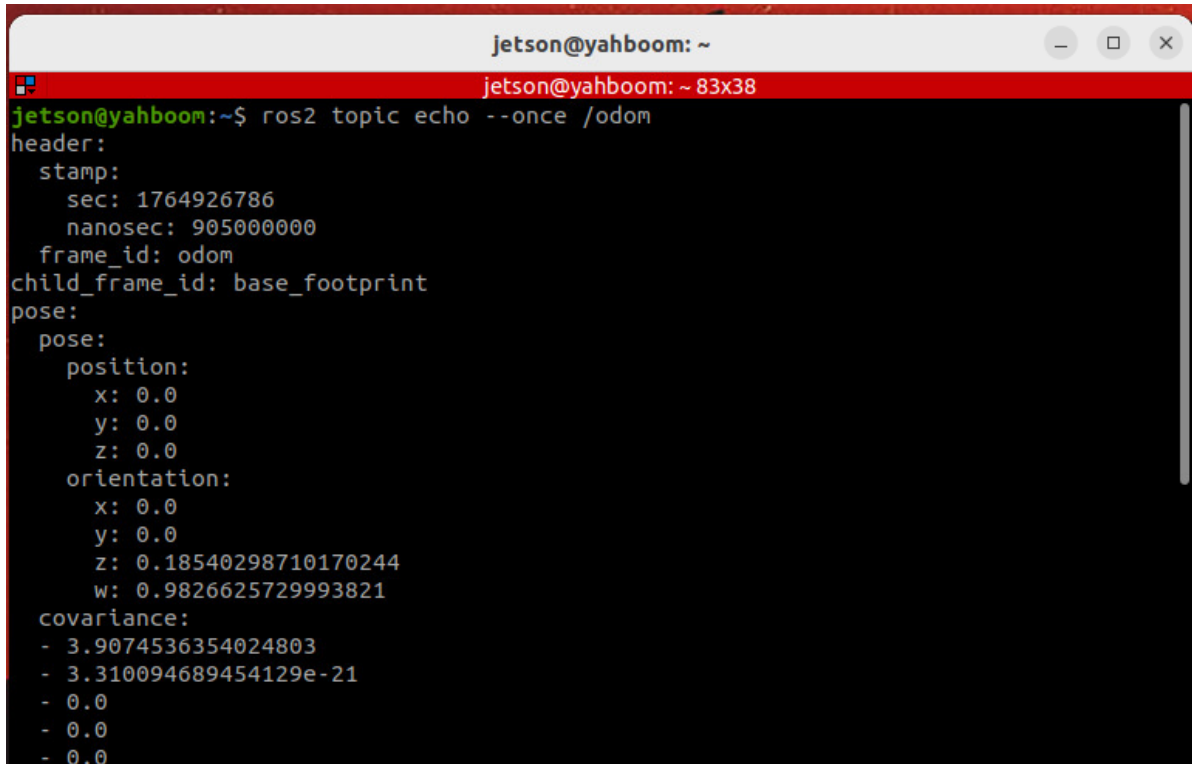
- In robotics, odometry is also known as local positioning. Common types of odometry include wheel, laser, visual, and inertial odometry. The ROSMASTER-M3 chassis uses wheel odometry.

## 3.2 View Fused Odometry Data

- Enter the command in the terminal:

```
ros2 topic echo --once /odom
```

- `header`: Describes the data frame information.
- `pose`: Describes the robot's pose information, consisting of position and orientation information.
- `covariance`: Describes the covariance matrix of the data.



```
jetson@yahboom: ~  
jetson@yahboom: ~ 83x38  
jetson@yahboom:~$ ros2 topic echo --once /odom  
header:  
  stamp:  
    sec: 1764926786  
    nanosec: 905000000  
  frame_id: odom  
child_frame_id: base_footprint  
pose:  
  pose:  
    position:  
      x: 0.0  
      y: 0.0  
      z: 0.0  
    orientation:  
      x: 0.0  
      y: 0.0  
      z: 0.18540298710170244  
      w: 0.9826625729993821  
covariance:  
  - 3.9074536354024803  
  - 3.310094689454129e-21  
  - 0.0  
  - 0.0  
  - 0.0
```

## 4. Source Code Explanation

- `car_base.launch.py` path: Starts the laser fusion and fused odometry localization node.

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/common/car_base.launch.py
```

- This launch file mainly starts the laser fusion node and the fused odometry node, and prints the startup log.

```
def generate_launch_description():  
  
    # Laser radar fusion and filtering startup file  
  
    bringup_laser_launch=IncludeLaunchDescription(PathJoinSubstitution([FindPackages  
here('m3_bringup'),'launch','common', 'bringup_laser.launch.py']))  
    # Multi-odometry fusion positioning startup file
```

```

merge_odom_launch=IncludeLaunchDescription(PathJoinSubstitution([FindPackageShare(
'm3_bringup'),'launch', 'common','merge_odom.launch.py']))

    return LaunchDescription([
        bringup_laser_launch,      # Laser radar fusion and filtering startup
file
        TimerAction(
            period=2.0,
            actions=[
                merge_odom_launch,
            ]
        ),

        LogInfo(msg=[Fore.GREEN+'=====Robot Base
Start====='], Fore.RESET)),
    ])

```

- `merge_odom.launch.py` path: Starts the fused odometry localization node.

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/common/merge_odom.launch.py
```

- This launch file starts the `ekf_filter_node` from the `robot_localization` package for multi-sensor fusion localization and finally prints the startup log.

```

def generate_launch_description()->LaunchDescription:

    localization_config_file=os.path.join(get_package_share_directory('m3_bringup')
, 'config', 'robot_localization.yaml')

    # Declare arguments
    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            "namespace",
            default_value="",
            description="Add namespace",
        )
    )

    # Initialize Arguments
    namespace=LaunchConfiguration("namespace")

    return LaunchDescription([
        *declared_arguments,
        LogInfo(
            msg=[Fore.GREEN+'=====Merge_Odom Localization
Start=====\\n', Fore.RESET]),

```

```

Node(
  package='robot_localization',
  executable='ekf_node',
  name='ekf_filter_node',
  namespace=namespace,
  output='screen',
  parameters=[localization_config_file],
  remappings=[('/odometry/filtered', '/odom')]
),
])

```

## 5. Fused Odometry Parameters

Parameter file path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/robot_localization.yaml
```

- Here, the sensor fusion uses the wheel odometry and IMU angular velocity for fusion. See the comments for the specific meaning of the parameters.

```

### ekf config file
#
###
ekf_filter_node:
  ros__parameters:

    use_sim_time: false
    frequency: 25.0                # Fused odometry output
frequency
    sensor_timeout: 0.3            # Sensor timeout, the minimum
frequency at which the filter generates new output, determined by the sensor
frequency
    two_d_mode: true               # 2D position state estimation,
false enables 3D state estimation
    transform_time_offset: 0.0     # Time offset provided by the
ekf_localization_node for generated transforms
    transform_timeout: 0.0         # Specifies the TF listener
timeout
    print_diagnostics: false       # Output debugging information
    debug: false                  # Output debugging information
to a file
    print_diagnostics: false       # Output log to diagnostics
topic, only enable for debugging
    debug_out_file: /path/to/debug/file.txt # Debug file output path, only
effective when debug is true
    publish_tf: true              # whether to publish TF
transforms
    permit_corrected_publication: false # whether to allow old sensor
data to trigger a state update and then republish the result
    publish_acceleration: false    # whether to publish the
acceleration state (accel field in nav_msgs/Odometry)

```

```

    map_frame: map                                # Defaults to "map" if
unspecified
    odom_frame: odom                              # Defaults to "odom" if
unspecified
    base_link_frame: base_footprint               # Robot's base coordinate frame
(fixed to the robot)
    world_frame: odom                             # The filter's world coordinate
frame, used to define the reference frame for state estimation

    # wheel odometry (provided by the chassis wheel motor encoders),
frequency is about 11hz
    odom0: odom_raw
    # Test 1
    odom0_config: [false, false, false,
                  false, false, false,
                  true, true, false,
                  false, false, true,
                  false, false, false]

    odom0_queue_size: 5                          # Queue size for subscribing to
the odom0 topic
    odom0_nodelay: false
    odom0_differential: false                    # whether to convert absolute
pose data to relative velocity (via differential calculation) for fusion.
    odom0_relative: true                        # whether to use the sensor's
first data as the "zero point", with subsequent data participating in fusion as
relative values.
    odom0_pose_use_child_frame: false            # whether to use the initial
pose of the child_frame_id in the sensor message as the origin, suitable for
scenarios where the sensor has a fixed offset from base_link
    # odom0_pose_rejection_threshold: 20.0 # Mahalanobis distance threshold
for pose data, values exceeding this are considered outliers and ignored.
    # odom0_twist_rejection_threshold: 1.5 # Mahalanobis distance threshold
for twist (velocity) data.

    # IMU data, raw data provided by the chassis ROS control board, filtered
data is published, frequency is about 25hz
    imu0: imu/data_raw
    # Test 1 (works well, yaw angle is stable during continuous rotation,
will try to debug and calibrate IMU separately later)
    imu0_config: [false, false, false,
                  false, false, false,
                  false, false, false,
                  false, false, true,
                  false, false, false]

    imu0_nodelay: false
    imu0_differential: false
    imu0_relative: false
    imu0_queue_size: 5
    # imu0_pose_rejection_threshold: 20.0          # Note the
difference in parameter names # 0.8
    # imu0_twist_rejection_threshold: 1.542        # 0.8
    # imu0_linear_acceleration_rejection_threshold: 10.0 #
    imu0_remove_gravitational_acceleration: true    # whether to
remove the gravitational component from the IMU's acceleration data. Enable if
the IMU does not automatically remove gravity.

```



```

    # Predict based on control commands
    use_control: false                                # Whether to use
control input (e.g., cmd_vel) in the prediction phase to generate an acceleration
term to assist state prediction.
    stamped_control: false                            # Specifies the message
type of the control input. true for geometry_msgs/TwistStamped (with timestamp),
false for geometry_msgs/Twist (without timestamp).
    control_timeout: 0.2                              # Timeout for control
commands (in seconds). If no new command is received within this duration, the
filter will stop using control input for prediction.
    control_config: [true, false, false, false, false, true] # Specifies
which velocity dimensions use control input, in the order: [vx, vy, vz, vroll,
vpitch, vyaw]. true enables control input for that dimension.
    acceleration_limits: [1.5, 0.0, 0.0, 0.0, 0.0, 3.4]    # Maximum
acceleration limits for each velocity dimension (in units of velocity/second,
e.g., m/s2 or rad/s2).
    deceleration_limits: [1.5, 0.0, 0.0, 0.0, 0.0, 4.5]    # Maximum
deceleration limits for each velocity dimension (maximum rate of change opposite
to the direction of acceleration).
    deceleration_gains: [0.9, 0.9, 0.0, 0.0, 0.0, 1.0] # Smoothing factor
for acceleration/deceleration (between 0 and 1), used to simulate the robot's
inability to instantly reach maximum acceleration/deceleration (e.g., due to
inertia).
    # The closer the value is to 1, the closer the acceleration/deceleration
is to the set limit. The closer the value is to 0, the smoother the change (and
the greater the delay).

#=====
# Sensor covariance. It is recommended to use the default values without
professional measurement equipment for calibration.
#
#=====
# Defines the process noise covariance matrix (15x15) for the filter's
prediction model. The larger the value, the lower the filter's confidence in the
prediction, and the more it relies on sensor data.
    process_noise_covariance: []
    # The covariance matrix for the initial state estimate (15x15). The
larger the value, the lower the confidence in the initial state.
    initial_estimate_covariance: []

```